# Final Project Report

## Comparing LeGEND and scenoRITA for Collision-Related Autonomous Driving Scenario Testing

SWE 295: Software Testing and Debugging for Autonomous Driving

**Prepared by:**
Khundrakpam Partha

**Instructor:**
Professor Joshua Garcia

University of California, Irvine June 2026

# Introduction

Autonomous driving systems rely on multiple connected modules, including perception, prediction, planning, and control. Among these modules, the planning module is especially important because it decides how the ego vehicle should move in response to surrounding vehicles, pedestrians, lanes, traffic rules, and possible hazards. Even when an autonomous driving system works correctly in many normal cases, rare or complex traffic situations can still expose safety-critical failures such as collisions, unsafe lane changes, or hard braking. Because of this, simulation-based testing has become an important method for evaluating autonomous driving systems before they are deployed in the real world.

In this project, I focused on scenario-based testing for autonomous driving systems, especially testing methods that can expose collision-related planning failures. My original goal was to design an AI-assisted workflow that could generate, rank, or prioritize test scenarios for finding planning-module bugs. However, as I studied the existing research tools more closely, I found that several tools already support automated scenario generation in different ways. Because of this, I revised my goal from building a completely new AI-assisted tool to evaluating and comparing existing tools that approach scenario generation differently.

The two main tools I compared were scenoRITA and LeGEND. scenoRITA is a search-based scenario generation tool that creates and mutates synthetic test scenarios in simulation. It explores a wide scenario space by changing obstacle properties such as position, speed, type, size, and trajectory. LeGEND, on the other hand, is an AI-assisted tool that starts from real-world accident reports. It uses language-model-based incident pattern synthesis to convert textual accident descriptions into structured simulation scenarios. This makes LeGEND different from scenoRITA because its scenarios are grounded in real accident descriptions rather than only generated through synthetic mutation.

The main purpose of this report is to compare how these two tools perform under a similar runtime budget. I ran LeGEND and scenoRITA for approximately 6.5 hours and compared their results using collision count, scenario throughput, collision rate, violation diversity, interpretability, and execution stability. This comparison is not meant to prove that one tool is universally better than the other. Instead, it shows the tradeoff between two different approaches: search-based exploration and real-accident-based AI scenario generation. Through this comparison, I evaluate how each tool contributes to autonomous driving system testing and how the results connect to my learning goals in this course.

# Project Goals and Change in Direction

At the beginning of the quarter, my project was organized around four learning goals.

**GOAL 1:-** The first goal was to understand how autonomous driving software is structured and where failures commonly occur during simulation-based testing. For this goal, I focused on Apollo rather than Autoware. I studied the general Apollo autonomous driving pipeline, especially the relationship between perception, prediction, planning, and control, and focused on how planning-related failures can appear during simulation. This helped me understand why scenario-based testing is important for finding safety-critical behaviors such as collisions, hard braking, and unsafe maneuvers.

**GOAL 2:-** My second goal was to learn and apply scenario-based testing techniques using tools such as scenoRITA and Doppelgänger/DoppelTest to generate failure-inducing test cases. I worked directly with these tools and tested them in a simulation environment. In particular, I used scenoRITA to generate and evaluate a large number of scenarios and studied its violation outputs, including collision-related results. I also explored DoppelTest and reviewed how it connects to scenario-based testing and failure discovery. This goal helped me move from only reading about autonomous driving testing to actually running tools and inspecting their outputs.

## CHANGES APPLIED

**GOAL 3:-** My third goal was to design and prototype an AI-assisted tool that generates or prioritizes test scenarios to expose bugs in ADS planning modules. This goal changed during the project. As I reviewed more research, I found that there are existing tools that already use AI-assisted methods for scenario generation. So with the help of professor and our guide, i choose LeGEND which uses language-model-based incident pattern synthesis to convert real-world accident reports into simulation scenarios. Because of this, I shifted my goal from building a completely new AI-assisted prototype to evaluating and comparing an existing AI-assisted tool against a search-based scenario generation tool.

My updated Goal 3 became to compare LeGEND and scenoRITA under a similar 6.5-hour runtime budget, with collision detection as the main evaluation metric. I also considered scenario throughput and execution stability as supporting factors because they help explain how many scenarios each tool was able to run and how reliable the tools were during testing.

This change does not mean that Goal 3 was abandoned. Instead, it became more research-focused and better connected to the existing tool landscape. Rather than creating a new tool without first

understanding what current tools can already do, I used LeGEND as an example of an AI-assisted scenario generation approach and compared it with scenoRITA's search-based approach. This allowed me to evaluate the strengths and limitations of AI-assisted testing in a more grounded way.

**GOAL 4:-** My fourth goal was to improve my skills in searching for and reviewing software engineering research. I addressed this goal by reading and connecting multiple papers related to autonomous driving testing, including scenoRITA, DoppelTest, LeGEND, and SoVAR. These papers helped me understand different approaches to scenario generation, accident-based testing, trajectory reconstruction, and failure analysis. Reviewing these papers also helped me refine the direction of my project and improve my research reading skills and evaluations.

Overall, I addressed all four learning goals. Goal 1 was achieved through studying Apollo and planning-related simulation failures. Goal 2 was achieved through hands-on work with scenario-based testing tools such as scenoRITA and DoppelTest. Goal 3 was revised from building a new AI-assisted prototype to evaluating an existing AI-assisted scenario generation tool, LeGEND, against scenoRITA. Goal 4 was achieved through reading and analyzing multiple research papers and connecting them to my experimental work.

# Background and Related Work

## Apollo and Simulation-Based ADS Testing

For the hands-on part of this project, I focused on Apollo rather than Autoware. Apollo is an open autonomous driving platform that includes major ADS modules such as localization, perception, prediction, planning, and control (Fan et al., 2018). In this project, I focused especially on the planning side of the pipeline because the planner is responsible for deciding how the ego vehicle should move in response to surrounding vehicles, obstacles, lanes, and traffic rules.

Simulation-based testing is important because many safety-critical failures are rare, difficult to reproduce, or unsafe to test directly on public roads. By using simulation, researchers can repeatedly execute traffic scenarios and observe whether the autonomous vehicle produces unsafe behavior such as collisions, hard braking, or unsafe lane changes. In my project, collision detection became the main evaluation focus because collisions are direct safety-critical failures and are easier to compare across tools than broader qualitative behaviors.

This background is important because both scenoRITA and LeGEND generate scenarios to test autonomous driving systems in simulation, but they approach scenario generation differently. scenoRITA uses search-based scenario generation, while LeGEND uses accident-report-based AI scenario generation. The rest of this section explains the tools and research papers I studied before presenting my experiment.

## scenoRITA

scenoRITA is one of the main scenario-based testing tools I studied and used in this project. It is designed to generate diverse test scenarios for autonomous driving systems by creating and mutating scenario elements in simulation (Huai et al., 2023a). Instead of starting from a real-world accident report, scenoRITA begins from a simulation map and generated scenario genes. It can mutate obstacle properties such as obstacle location, type, speed, size, and trajectory (Huai et al., 2023a).

For my project, scenoRITA was important because it provided a search-based approach to finding failure-inducing scenarios. This made it a useful comparison point against LeGEND, which follows a different approach by starting from textual accident reports. In my final experiment, scenoRITA was used to evaluate how many collision-related failures could be found within a fixed runtime budget.

## DoppelTest / Doppelgänger

I also studied and tested DoppelTest as part of my second learning goal on scenario-based testing. DoppelTest is different from many simulation-based ADS testing tools because it does not rely only on simple scripted obstacles. Instead, it introduces the idea of “doppelgänger” testing, where multiple instances of the same ADS are executed in the same scenario and each instance controls a different autonomous vehicle (Huai et al., 2023b). This means the other vehicles in the scenario are not just basic obstacles with fixed behavior; they are also controlled by the ADS and can react to traffic rules, lanes, and surrounding participants.

This design is important because it makes the discovered violations more meaningful. In many ADS testing tools, a collision can happen because an obstacle behaves unrealistically or violates traffic rules, which makes it harder to blame the ADS under test. DoppelTest tries to reduce this problem by making every vehicle an ADS-controlled vehicle. Therefore, when a collision or traffic-rule violation occurs, the result is more likely to reveal a real weakness in the ADS decision-making process rather than only an unrealistic obstacle behavior (Huai et al., 2023b).

## LeGEND

LeGEND was the main AI-assisted scenario generation tool I studied for the revised third goal. Unlike search-based tools that begin from a simulation map and mutate scenario parameters, LeGEND uses a top-down scenario generation process. It starts from abstract functional scenarios, then transforms them into logical scenarios, and finally searches for concrete critical scenarios that can be executed in simulation (Tang et al., 2024). This structure was important for my project because it showed that AI can be used not only to generate random test cases, but also to transform human-readable accident descriptions into structured ADS test scenarios.

A key reason I chose LeGEND is that it uses real-world accident reports as the source of its functional scenarios. The paper explains that this helps avoid unrealistic functional scenarios because the high-level scenario comes from an actual accident description rather than from purely synthetic generation (Tang et al., 2024). LeGEND then uses LLMs to extract relevant information from the textual accident report and convert that information into a formal logical scenario. In this way, LeGEND provided a strong comparison point against scenoRITA: scenoRITA explores a wide synthetic search space, while LeGEND focuses on accident-report-based scenario generation.

For my project, LeGEND was useful because its output is more traceable than a normal telemetry-only result. The output preserves the original accident report, the interpreted functional scenario, the structured scenario dictionary, and the generated logical scenario. This made it possible to see how a real-world crash description was transformed into a simulation test case. Because of this, I used LeGEND as the AI-assisted tool in my final comparison with scenoRITA**.**

## SoVAR

I also reviewed SoVAR to understand another approach to accident-based autonomous driving testing. The most useful idea and understanding I got from SoVAR was its focus on analyzing the behavior of the ego vehicle and the surrounding vehicle during collision cases. In particular, the paper discusses examining ego and NPC vehicle trajectories to understand the actions that led to safety violations(Guo et al., 2024). Although I did not use SoVAR as one of the final experimental tools, reading it helped me think beyond raw collision counts and consider how trajectory or behavior information can support deeper failure analysis.

# Experimental Setup

The experiment was organized in the following steps:

**Step 1: Select the tools for comparison.**

I selected LeGEND and scenoRITA because they represent two different approaches to ADS scenario generation. LeGEND represents an AI-assisted, accident-report-based approach, while scenoRITA represents a search-based synthetic scenario generation approach [LeGEND citation; scenoRITA citation].

**Step 2: Use collision detection as the main metric.**

The main evaluation metric was the number of detected collisions. I chose collision detection because collisions are direct safety-critical failures and both tools produced outputs that could be used to count collision-related results.

**Step 3: Run LeGEND.**

I attempted to run LeGEND on all 18 available accident reports. However, the run completed 16 reports before encountering segmentation fault errors. The final LeGEND run lasted approximately 6.5 hours, so I used that runtime as the comparison budget for scenoRITA.

**Step 4: Run scenoRITA with a similar runtime budget.**

I then ran scenoRITA for approximately the same 6.5-hour time period. This allowed me to compare both tools under a similar runtime condition instead of comparing a completed dataset from one tool against a longer or shorter run from the other tool.

**Step 5: Count scenarios and collision results.**

For LeGEND, I counted executed scenarios using the iteration markers in the run log and counted collisions from the JSON result files. For scenoRITA, I counted executed scenarios from the records folder and counted collision records from the Collision.csv output file. These counts were then used to compare scenario throughput, raw collision count, and collision rate per scenario.

*The guiding question for this experiment was:* **Under a similar 6.5-hour runtime budget, does a real-accident-based AI scenario generation tool or a search-based scenario generation tool find more collision-related ADS planning failures?**

| Setup Item | LeGEND | scenoRITA |
|---|---|---|
| **Tool type** | AI-assisted, accident-report-based scenario generation | Search-based synthetic scenario generation |
| **Scenario source** | Real-world accident reports | Simulation map and generated scenario genes |
| **Runtime target** | Approximately 6.5 hours | Approximately 6.5 hours |
| **Main metric** | Number of detected collisions | Number of detected collision records |
| **Supporting metrics** | Scenario throughput and execution stability | Scenario throughput and execution stability |
| **Main output used** | Log files and JSON result files | Collision.csv and scenario records folder |

This setup allowed me to compare both tools using the same time budget while still acknowledging that they generate scenarios in different ways.

# Result

The results show that scenoRITA evaluated more scenarios than LeGEND within a similar runtime budget. LeGEND ran for approximately 6 hours and 22 minutes and evaluated about 1,610 scenarios, while scenoRITA ran for approximately 6 hours and 32 minutes and evaluated 3,020 scenarios. This means scenoRITA evaluated about 1.9 times more scenarios than LeGEND during the experiment.

| Metric | LeGEND | scenoRITA |
|---|---|---|
| **Runtime** | 6h22m / approx. 6.5 hrs | 6h32m / approx. 6.5 hrs |
| **Scenarios evaluated** | 1,610 | 3,020 |
| **Raw collisions** | 6 | 9 |
| **Collision rate** | 0.37% | 0.30% |
| **Collisions per hour** | 0.92 | 1.38 |

In terms of raw collision count, scenoRITA detected more collision records. scenoRITA found 9 collision records, while LeGEND found 6 collisions. This means scenoRITA produced more total collision instances within the fixed runtime budget.

# Output Interpretation

The result numbers show how many collisions each tool found, but the output format also matters because it affects how easy it is to understand each failure. In this project, scenoRITA and LeGEND produced very different kinds of evidence.

scenoRITA's collision output was mainly telemetry-focused. Its Collision.csv file recorded the scenario ID and the ego vehicle state at the collision moment, including the ego vehicle's position, heading, and

speed. It also recorded the obstacle or NPC vehicle position, heading, type, length, and width. This made scenoRITA useful for identifying exactly where and under what vehicle-state conditions the collision happened.

| scenoRITA Output Field | Meaning |
|---|---|
| **sce_id** | Scenario ID where the collision happened |
| **ego_x, ego_y** | Ego vehicle position at the collision moment |
| **ego_theta** | Ego vehicle heading or direction |
| **ego_speed** | Ego vehicle speed during collision |
| **obs_x, obs_y** | Obstacle/NPC position at collision moment |
| **obs_type** | Type of obstacle/NPC, stored as a numeric category |
| **obs_theta** | Obstacle heading or direction |
| **obs_length, obs_width** | Size of the obstacle |

LeGEND's output was more scenario-traceability focused than scenoRITA's telemetry-style CSV output. In the LeGEND JSON result file, the output included the collision count and the critical scenarios that triggered collisions. These critical scenarios showed concrete Python-style scenario parameters, such as vehicle lane, offset, initial speed, acceleration or deceleration behavior, lane-change behavior, target

speed, and trigger sequence. This made LeGEND useful because I could inspect not only whether a collision occurred, but also the generated scenario logic that led to the collision.

| LeGEND Output Field | Meaning |
|---|---|
| **accident_report** | Original real-world crash report text |
| **functional_scenario** | LeGEND/LLM's summarized version of the accident situation |
| **func_scenario_dict** | Structured version of the scenario, such as road structure and initial actions |
| **candidate_ego** | Vehicle selected as the ego vehicle |
| **logical_scenario** | Formal test case object created from the accident report |
| **collision_num** | Number of collisions produced by the generated scenario |
| **critical_scenarios** | Critical/collision-triggering scenario parameters, if found |

In contrast, scenoRITA's collision output focused more on the vehicle state at the collision moment, such as ego position, ego speed, obstacle position, obstacle type, and obstacle size. Therefore, scenoRITA was stronger for telemetry-based collision measurement, while LeGEND was stronger for understanding the generated scenario setup that produced the collision.

# Analysis and Discussion

The results show that scenoRITA was stronger in raw collision discovery under the fixed runtime budget. It evaluated 3,020 scenarios and found 9 collision records, while LeGEND evaluated about 1,610 scenarios and found 6 collisions. This suggests that scenoRITA's search-based approach was able to explore more test cases in the same general time period and produce more total collision records.

However, the collision rate per scenario shows a more balanced result. LeGEND found 6 collisions from about 1,610 evaluated scenarios, giving a collision rate of 0.37%. scenoRITA found 9 collision records from 3,020 evaluated scenarios, giving a collision rate of 0.30%. This means that even though LeGEND found fewer total collisions, its accident-report-based scenarios produced slightly more collisions relative to the number of scenarios executed.

This result suggests an important tradeoff between the two tools. scenoRITA appears stronger for high-throughput search-based exploration because it can generate and evaluate many synthetic scenarios. This makes it useful when the goal is to search broadly for failure-inducing cases. LeGEND appears stronger in collision density and scenario traceability because its scenarios are grounded in real accident reports and its output shows the generated scenario logic that led to the collision.

Another important point is that raw collision count should not be interpreted as the only measure of tool effectiveness. In the scenoRITA output, several collision records occurred around similar coordinates. This suggests that some records may represent repeated versions of the same failure area rather than completely unique failures. Therefore, scenoRITA's 9 collision records are useful evidence of collision discovery, but they should not automatically be treated as 9 distinct planning bugs.

Overall, the experiment shows that scenoRITA was better at producing more collision records within the runtime budget, while LeGEND produced fewer but slightly more collision-dense and more traceable results. Because the tools use different scenario-generation strategies, the result should be interpreted as a comparison of their practical behavior under the same time constraint, not as final proof that one tool is universally better than the other.

# Limitations

This experiment has several limitations. First, the comparison was runtime-bounded rather than a perfect full-dataset comparison. I attempted to run LeGEND on all 18 available accident reports, but the

execution completed 16 reports before encountering segmentation fault errors. Because of this, the LeGEND result should be interpreted as an adapted local run rather than a completely clean execution of the original tool.

Second, LeGEND and scenoRITA do not explore the same scenario space. LeGEND generates scenarios from real-world accident reports, while scenoRITA generates and mutates synthetic scenarios in simulation. Therefore, the tools are not being compared on identical inputs. The experiment compares how each tool behaved under a similar runtime budget, but it does not prove that one scenario generation method is universally better than the other.

Third, collision count alone is not enough to fully evaluate tool effectiveness. scenoRITA found 9 collision records, but several of the collisions appeared near similar coordinates. This suggests that some collision records may represent repeated versions of the same failure area rather than completely unique planning failures. Because of this, raw collision count should be interpreted together with collision location, scenario diversity, and repeated failure patterns.

Fourth, this experiment was based on one main controlled run for each tool. More repeated runs would be needed to make a stronger statistical comparison. Additional experiments with longer runtime budgets, multiple maps, and repeated trials could help determine whether the observed pattern remains consistent across different testing conditions.

Overall, these limitations mean that the results should be interpreted carefully. The experiment provides useful evidence about how LeGEND and scenoRITA behaved in my local setup under a fixed runtime budget, but it should not be treated as a final or complete benchmark of both tools.

# Self-Assessment and Expected Grade

I believe I should at least get an A- or A for this project. I was able to address the main goals from my original learning plan. I studied the Apollo autonomous driving pipeline, focused on planning-related simulation failures, applied scenario-based testing tools such as scenoRITA and DoppelTest, reviewed multiple ADS testing papers, and completed a hands-on comparison between scenoRITA and LeGEND. The course requirement was to use at least three research papers, and I used that requirement by reviewing and connecting four papers: LeGEND, scenoRITA, DoppelTest, Apollo and SoVAR.

The strongest part of my project is that I did not only summarize the papers. I installed and ran real ADS testing tools, collected outputs from long-running experiments, counted scenario executions, analyzed collision results, and compared two different scenario generation approaches. My final comparison showed that scenoRITA found more raw collision records under the fixed runtime budget, while LeGEND produced a slightly higher collision rate per scenario and more traceable scenario outputs.

At the same time, I recognize that there were limitations in the project. I originally wanted to run a longer experiment, but LeGEND encountered repeated segmentation fault errors. Because of this, only 16 out of the 18 available accident reports were completed, and the intended longer runtime was reduced to approximately 6.5 hours. This issue limited the completeness of the LeGEND run, and I think it is reasonable that some points could be deducted for this technical limitation.

However, I also believe the issue does not invalidate the project because I reported it clearly, treated the experiment as a runtime-bounded comparison, and did not overclaim the results. The segmentation fault is also an important future research task. As I continue this research, I plan to debug this issue further so that LeGEND can complete all reports and support longer controlled runs.

Overall, I believe an A- is appropriate because I achieved the main learning goals, adapted my project direction based on the research landscape, completed hands-on experiments, produced measurable results, and critically analyzed the limitations of my work.

# Future Work

Future work could improve this comparison by running both tools for longer and more controlled time periods, such as 12-hour or 24-hour runs. Repeating each experiment multiple times would also make the comparison stronger because it would show whether the collision counts are consistent across runs or affected by randomness in the scenario generation process.

Another useful direction would be to compare unique collision clusters instead of only raw collision count. In my scenoRITA output, some collision records occurred around similar coordinates, so future analysis could group collision records by location, vehicle state, or scenario pattern. This would make it easier to separate repeated versions of the same failure area from truly different planning failures.

Future work could also test the tools across more maps and road environments. The LeGEND paper evaluated the tool using Apollo with LGSVL and selected the San Francisco HD map because it included

road structures that matched the accident-report requirements. Testing on additional maps, if supported, would help evaluate whether the results remain similar across different road structures and environments.

Finally, future work could combine the strengths of both approaches. scenoRITA could be used for broad search-based exploration, while LeGEND could be used to generate more realistic accident-inspired scenarios. A combined workflow could first use accident reports to create realistic high-level scenario patterns, then use search-based mutation to explore more concrete variations of those scenarios.

# Conclusion

In this project, I studied scenario-based testing for autonomous driving systems with a focus on collision-related planning failures in Apollo. My original learning goals were to understand ADS structure, apply scenario-based testing tools, design or evaluate an AI-assisted testing workflow, and improve my ability to review software engineering research. Through this project, I was able to address all four goals.

The main experiment compared LeGEND and scenoRITA under a similar fixed runtime budget. scenoRITA evaluated more scenarios and found more raw collision records, while LeGEND found fewer total collisions but had a slightly higher collision rate per scenario. This result shows an important tradeoff between the two approaches. scenoRITA is useful for broad search-based exploration because it can generate and execute many synthetic scenarios. LeGEND is useful for accident-report-based testing because its results are more traceable to generated scenario logic and real-world crash patterns.

My third goal changed during the project, but it was not abandoned. Instead of building a new AI-assisted tool from scratch, I shifted to evaluating LeGEND as an existing AI-assisted scenario generation tool and comparing it with scenoRITA. This change made the project more grounded in current research and helped me better understand what AI-assisted testing tools can already do.

Overall, this project gave me hands-on experience with ADS testing tools, long-running simulation experiments, output analysis, and research comparison. Although the experiment had limitations, such as LeGEND's segmentation fault and the fact that both tools do not explore the same scenario space, the final comparison still provided meaningful insight into how different scenario generation strategies can expose collision-related ADS planning failures.

## References

Tang, S., Zhang, Z., Zhou, J., Lei, L., Zhou, Y., & Xue, Y. (2024). *LeGEND: A Top-Down Approach to Scenario Generation of Autonomous Driving Systems Assisted by Large Language Models*.

Huai, Y., Almanee, S., Chen, Y., Wu, X., Chen, Q. A., & Garcia, J. (2023). *scenoRITA: Generating diverse, fully mutable, test scenarios for autonomous vehicle planning*. IEEE Transactions on Software Engineering, 49(10), 4656–4676. https://doi.org/10.1109/TSE.2023.3309610

Huai, Y., Chen, Y., Almanee, S., Ngo, T., Liao, X., Wan, Z., Chen, Q. A., & Garcia, J. (2023). *Doppelgänger test generation for revealing bugs in autonomous driving software*. In Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE) (pp. 2591–2603)

Guo, A., Zhou, Y., Tian, H., Fang, C., Sun, Y., Sun, W., Gao, X., Luu, A. T., Liu, Y., & Chen, Z. (2024). *SoVAR: Building generalizable scenarios from accident reports for autonomous driving testing*. ASE 2024.

Fan, H., Zhu, F., Liu, C., Zhang, L., Zhuang, L., Li, D., Zhu, W., Hu, J., Li, H., & Kong, Q. (2018). *Baidu Apollo EM motion planner*. arXiv